Programación asistida por IA en Emacs (Python)

Guía y ejercicios

Luis A. Medina

2026-10-26

Índice

1. Lo mínimo de Emacs 1
2. El ciclo de 60 segundos 2
3. Preparación 3
4. Comandos de IA (`C-c I`) 3
5. Comandos de Python (`C-c P`) 5
6. Refactorizar con diff 7
7. Recetas de prompt 7
8. Reglas de precisión 8
9. Mini-proyecto de práctica 8
10. Ejercicios 9
11. Git y GitHub con Magit 12
12. Proyectos: agente de código con DeepSeek 16
13. Chuleta de una página 18
14. Problemas comunes 18
Para ir más lejos 20
15. Repetir la guía desde cero 20

> **Antes de empezar:** instala el entorno siguiendo `docs/instalacion.md` y abre Emacs con `emacs --init-directory .../emacs`.

Esta guía asume que **nunca has usado Emacs en serio**. No hace falta saber Lisp ni configurar nada: los comandos ya están instalados y verificados. Al final vas a tener un ritmo de trabajo repetible: escribir un test, pedirle el código a la IA, aplicarlo, formatearlo y comprobarlo, todo sin salir de Emacs.

> **Aviso:** La idea central: la IA no escribe el programa por ti. Tú controlas el ciclo; la IA rellena los huecos y tú verificas con tests.

1. Lo mínimo de Emacs

Si ya usas Emacs, salta a la sección 2. Si no, estos diez conceptos bastan para toda la guía.

Concepto	Qué es	Cómo se usa
Buffer	Un archivo abierto en memoria	`C-x C-f` abre uno; `C-x C-s` guarda

Concepto	Qué es	Cómo se usa
Región	El texto seleccionado	`C-SPC` marca el inicio, mueves el cursor, el resto es región
Punto	Donde está el cursor	Es la posición actual
`M-x`	Ejecutar un comando por nombre	`M-x` es `Alt+x`; escribes el nombre y `RET`
`C-g`	Cancelar	Si algo se queda a medias, `C-g`
`C-S-_`	Deshacer	Repetir para deshacer más
`C-x h`	Seleccionar todo el buffer	Útil antes de pedir una revisión completa
`C-h k`	¿Qué hace esta tecla?	`C-h k` y luego la combinación
`C-c ...`	Prefijo de comandos	`C-c I` es el prefijo de IA; espera una segunda tecla
`C-c RET`	Enviar en un buffer de chat	Solo dentro de los buffers `*IA: ...*`

> **Ejemplo:** Regla mental: `C-c I` significa **I**A y `C-c P` significa **P**ython. La segunda tecla es la acción: `e` explicar, `t` tests, `f` formatear…

Seleccionar código

1. Pon el cursor al inicio de la función.
2. `C-SPC` (marca el inicio).
3. Mueve el cursor hasta el final de la función.
4. Ejecuta el comando (`C-c I e`, por ejemplo).

Si **no** hay selección, casi todos los comandos de IA usan el **buffer completo**. Para archivos grandes, selecciona solo la función: la respuesta será más rápida y precisa.

---

2. El ciclo de 60 segundos

Este es el corazón de la guía. Se repite hasta que el programa está terminado.

```
┌──────────────────────────────────────────────────┐
│  1. C-c P t      → tests en rojo (pytest)        │
│  2. C-c I i      → la IA implementa lo que falta │
│  3. pegar / diff → aplicar el código             │
│  4. C-c P f      → black (formato)               │
│  5. C-c P t      → tests en verde                │
│  6. C-c I r      → revisión rápida               │
│  7. commit                                       │
└──────────────────────────────────────────────────┘
```

Por qué funciona:

- El test es el contrato: define qué significa “correcto” antes de que la IA escriba nada.
- La IA solo rellena el hueco; no decide la arquitectura.

- `black` elimina discusiones de estilo.
- El test verde es la única señal de que puedes seguir.
- La revisión final detecta lo que el test no cubre.

Tiempo real: en un ejercicio pequeño, el ciclo completo tarda entre 30 y 90 segundos. La primera vez, más; la décima, menos.

---

3. Preparación

3.1 Verificar DeepSeek

Abre Emacs y ejecuta:

`M-x gptel-menu`

En el menú debe aparecer **DeepSeek** y el modelo **deepseek-flash**. También puedes cambiar directo:

Atajo	Backend
`C-c I 1`	DeepSeek Flash (rápido, por defecto)
`C-c I 2`	DeepSeek V4 Pro (más potente)
`C-c I 3`	Gemini 2.5 Flash

Si dice que falta la clave, revisa que exista `~/.emacs.d/.deepseek-key` con permisos `600`.

3.2 Verificar Python

En una terminal:

```
python3 -m pytest --version   # debe decir pytest 9.x
black --version               # debe existir
pyright --version             # LSP de Python
```

Los tres ya están instalados en tu máquina.

3.3 Abrir el proyecto de práctica

`C-x C-f ~/src/ia-python-practica/calc/operaciones.py RET`

Cuando abras un archivo `.py`, Emacs arranca **LSP + Pyright** automáticamente. La primera vez tarda unos segundos; verás el aviso LSP en la barra inferior. Espera a que termine antes de pedir diagnósticos.

---

4. Comandos de IA (`C-c I`)

Todos abren (o reutilizan) un buffer llamado `*IA: acción*`. La respuesta se escribe en vivo, como en un chat.

Atajo	Comando	Qué hace	Cuándo usarlo
`C-c I e`	`ia-explicar`	Explica el código: qué hace, qué recibe, qué devuelve, casos límite	No entiendes una función
`C-c I t`	`ia-tests`	Genera tests pytest (normales, bordes, errores)	Antes de implementar
`C-c I i`	`ia-implementar`	Implementa el código que hace pasar los tests	Tienes el test en rojo
`C-c I r`	`ia-revisar`	Revisión senior: bugs, casos límite, claridad, rendimiento	Antes de commit
`C-c I f`	`ia-refactor`	Refactoriza la **región** y muestra un **diff** acepta-ble/rechazable	El código funciona pero está feo
`C-c I d`	`ia-docstrings`	Añade docstrings (Google) y type hints	Antes de entregar
`C-c I x`	`ia-corregir`	Toma el error de `*compilation*` (pytest) o de flycheck y propone el arreglo	Un test falla y no sabes por qué
`C-c I a`	`ia-aceptar`	Acepta el diff pendiente de `f`	El diff te convence
`C-c I K`	`ia-rechazar`	Descarta el diff pendiente	El diff no te convence
`C-c I D`	`ia-diff`	Muestra el diff pendiente	Quieres revisar antes de aceptar
`C-c I c`	`ia-chat`	Chat libre con el backend activo	Preguntas sueltas
`C-c I m`	`gptel-menu`	Menú completo de gptel (modelo, directivas, contexto)	Ajustes finos
`C-c I s`	`gptel-send`	Envía el mensaje en un buffer de chat	Seguir la conversación
`C-c I 1`	`gptel-usar-deepseek`	Cambia a DeepSeek Flash	Rápido y barato
`C-c I 2`	`gptel-usar-deepseek-pro`	Cambia a DeepSeek V4 Pro	Tareas complejas
`C-c I 3`	`gptel-usar-gemini`	Cambia a Gemini 2.5 Flash	Alternativa

4.1 Explicar (`C-c I e`)

1. Abre `calc/operaciones.py`.
2. Selecciona `multiplicar` (o `C-x h` para todo).
3. `C-c I e`.
4. Lee la respuesta en `*IA: explicar*`.

La respuesta te dice que `multiplicar` suma: ese es el bug que arreglarás en el ejercicio 4.

4.2 Tests (`C-c I t`)

1. Abre `calc/operaciones.py`.
2. Selecciona `promedio`.
3. `C-c I t`.
4. Copia el test generado a `tests/test_operaciones.py` (o compáralo con el que ya existe).
5. `C-c P t` para ejecutarlo.

4.3 Implementar (`C-c I i`)

1. Abre `tests/test_operaciones.py`.
2. Selecciona `test_promedio` y `test_promedio_vacio`.
3. `C-c I i`.
4. Pega el código en `calc/operaciones.py`, sustituyendo los `raise NotImplementedError`.
5. `C-c P f` y `C-c P t`.

4.4 Corregir (`C-c I x`)

1. Ejecuta `C-c P t` y deja que falle.
2. Vuelve al archivo de código.
3. `C-c I x`.
4. La IA recibe las últimas líneas de `*compilation*` (el error real de pytest) y el código.
5. Aplica la corrección y vuelve a `C-c P t`.

> **Aviso:** Si `*compilation*` no existe o está vacío, `C-c I x` usa el error de flycheck en el punto o, en su defecto, la línea actual. Para mejores resultados, ejecuta primero `C-c P t`.

4.5 Conversar (`C-c I c` y `C-c RET`)

`C-c I c` abre `*IA: chat*`. Escribes tu pregunta al final y pulsas `C-c RET`. La respuesta se inserta debajo. Puedes seguir la conversación en el mismo buffer.

Para volver a enviar en cualquiera de los buffers `*IA: ...*`: `C-c RET`.

---

5. Comandos de Python (`C-c P`)

Atajo	Comando	Qué hace
`C-c P r`	`py-ejecutar`	Ejecuta el archivo actual con `python3`
`C-c P t`	`py-pytest`	Ejecuta `python3 -m pytest -q` desde la raíz del proyecto
`C-c P f`	`py-black`	Formatea el buffer con `black`
`C-c P l`	`py-lint`	Abre la lista de errores de Pyright/flycheck
`C-c P c`	`py-repl`	Abre un REPL de Python

5.1 Ejecutar (`C-c P r`)

Abre un script y pulsa `C-c P r`. La salida aparece en `*compilation*`. Si el script pide argumentos, usa `M-x compile` y edita la línea de comando.

5.2 Tests (`C-c P t`)

Busca la raíz del proyecto (`pyproject.toml`, `pytest.ini`, `setup.cfg` o `.git`) y ejecuta pytest ahí. Los errores son clicables: `RET` sobre un fallo te lleva a la línea.

Para ejecutar **un solo test**, usa `M-x compile` y escribe:

```
python3 -m pytest -q tests/test_operaciones.py::test_multiplicar
```

5.3 Formatear (`C-c P f`)

`black` reescribe el buffer. Si no quieres formatear todo, selecciona la región y usa `M-x blacken-region`.

5.4 Diagnóstico (`C-c P l`)

Abre `*Flycheck errors*` con los avisos de Pyright. Si está vacío, espera a que LSP termine o ejecuta `M-x lsp-workspace-restart`.

5.5 REPL (`C-c P c`)

Abre `*Python*`. Desde el archivo de código:

- `C-c C-c` envía **todo el buffer** al REPL.
- `C-c C-r` envía **la región**.
- `C-c C-z` salta al REPL y vuelve.

Esto es ideal para probar una función sin escribir un test, pero > **Aviso:** el test sigue siendo la fuente de verdad.

---

6. Refactorizar con diff

`C-c I f` es el comando más “mágico” y el que más disciplina exige.

1. Selecciona **solo** la función o el bloque que quieres cambiar.
2. `C-c I f`.
3. Espera: aparece un **overlay** con el código propuesto.
4. `C-c I D` abre el diff contra el original.
5. Decide:
   - `C-c I a` → aceptar y reemplazar.
   - `C-c I K` → rechazar y dejar el original.
6. `C-c P t` para confirmar que sigue verde.

   Ejemplo: El diff es tu red de seguridad: nunca aceptes un refactor sin mirarlo. Si algo sale mal, `C-/` deshace la aceptación.

Qué pedirle al refactor: nombres claros, funciones pequeñas, sin duplicación, sin cambiar el comportamiento. El comando ya incluye esa instrucción; si quieres otra, edita el prompt en `config.org` o usa `C-c I c` con un prompt libre.

---

7. Recetas de prompt

Copia y pega en `*IA: chat*` (o adáptalas a los comandos).

Explicar sin tocar nada

```
Explica esta función como si yo tuviera 5 años de Python.
No cambies el código. Dame: qué hace, qué recibe, qué devuelve
y un ejemplo de uso.
```

Tests primero

```
Escribe tests de pytest para esta función.
Incluye: caso normal, caso límite y error esperado.
Devuelve solo el archivo de test completo.
```

Implementar

```
Implementa esta función para que pasen estos tests.
Devuelve solo el código de producción, sin explicaciones.
No cambies los tests.
```

Refactorizar

```
Refactoriza sin cambiar el comportamiento.
Mejora nombres y divide en funciones pequeñas.
Devuelve solo el código reemplazado.
```

Corregir un error

```
Este es el error:
<pega el traceback>

Este es el código:
<pega la función>

Dame la causa en una línea y el parche mínimo.
```

Revisión

```
Revisa este código como un senior.
Ordena los hallazgos por gravedad.
Para cada uno: qué falla, por qué importa y el parche mínimo.
No reescribas todo el archivo.
```

No sé por qué falla

```
No me des código todavía. Dame 3 hipótesis de por qué falla,
ordenadas por probabilidad, y un experimento para distinguirlas.
```

---

8. Reglas de precisión

1. **Contexto siempre.** Archivo, función, error y qué esperabas. Sin eso, la IA adivina.
2. **Un cambio por vez.** Una función, un test, un commit. Los diffs grandes esconden errores.
3. **Tests después de cada cambio.** No acumules cambios sin verificar.
4. **No aceptes a ciegas.** Lee el diff. Si no lo entiendes, no lo aceptes.
5. **`C-/` es tu red.** Puedes deshacer la aceptación de un refactor.
6. **Si falla dos veces, cambia el prompt.** Dale el error exacto, no una descripción vaga.
7. **Nunca pegues secretos.** Claves, tokens y datos personales fuera del prompt.
8. **Revisa antes de commit.** `C-c I r` sobre el diff es más barato que depurar después.

   Importante: La IA es rápida, no infalible. La precisión viene del ciclo: test → cambio pequeño → verificación.

---

9. Mini-proyecto de práctica

Ruta: `~/src/ia-python-practica/`

```
ia-python-practica/
├── README.md
├── pyproject.toml
├── calc/
│   ├── __init__.py
│   └── operaciones.py
├── scripts/
│   └── reset.sh
└── tests/
    └── test_operaciones.py
```

9.1 Estado inicial

`calc/operaciones.py` tiene dos funciones sin implementar (`promedio`, `potencia`) y un bug intencional en `multiplicar` (suma en vez de multiplicar).

`tests/test_operaciones.py` ya trae 8 tests. Al ejecutar:

`C-c P t`

debes ver **4 fallos y 4 aprobados**. Ese es el punto de partida en rojo.

El repositorio tiene la etiqueta `estado-inicial` en ese commit. Para volver a este estado en cualquier momento:

```
cd ~/src/ia-python-practica
./scripts/reset.sh
```

Lo verás en detalle en la sección 15.

9.2 Abrir el proyecto

`C-x C-f ~/src/ia-python-practica/tests/test_operaciones.py RET`
`C-c P t`

Si pytest no encuentra `calc`, revisa que estés dentro de la carpeta del proyecto y que `pyproject.toml` tenga `pythonpath = ["."]`.

9.3 Archivos clave

`pyproject.toml`:

```toml
[project]
name = "ia-python-practica"
version = "0.1.0"
requires-python = "⩾3.10"

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
```

`calc/operaciones.py` (extracto):

```python
def multiplicar(a, b):
    # BUG intencional: debería multiplicar, no sumar.
    return a + b

def promedio(valores):
    raise NotImplementedError("Implementar en el ejercicio 3")

def potencia(base, exponente):
    raise NotImplementedError("Implementar en el ejercicio 3")
```

---

10. Ejercicios

Hazlos en orden. Cada uno tiene un criterio de “hecho” verificable. No pases al siguiente si el anterior no está verde.

Ejercicio 1 — Explicar (10 min)

Objetivo: perder el miedo al buffer de respuesta y entender el bug.

1. Abre `calc/operaciones.py`.
2. Selecciona `multiplicar` con `C-SPC` y el cursor.
3. `C-c I e`.
4. Lee la respuesta.

Hecho cuando: puedes decir en una frase qué hace mal `multiplicar`.

Ejercicio 2 — Tests primero (15 min)

Objetivo: generar tests y verlos fallar por la razón correcta.

1. Abre `calc/operaciones.py` y selecciona `promedio`.
2. `C-c I t`.
3. Copia el test generado a `tests/test_operaciones.py` (si añade casos que no tenías).
4. `C-c P t`.

Hecho cuando: los tests nuevos fallan con `NotImplementedError`, no con un error de sintaxis.

Ejercicio 3 — Implementar (15 min)

Objetivo: poner en verde `promedio` y `potencia`.

1. Abre `tests/test_operaciones.py` y selecciona `test_promedio`, `test_promedio_vacio` y `test_potencia`.
2. `C-c I i`.
3. Pega el código en `calc/operaciones.py`, sustituyendo los `raise NotImplementedError`.
4. `C-c P f`.
5. `C-c P t`.

Hecho cuando: solo queda fallando `test_multiplicar`.

Ejercicio 4 — Depurar (15 min)

Objetivo: usar el error real de pytest como contexto.

1. `C-c P t` y observa el fallo de `test_multiplicar`.
2. Vuelve a `calc/operaciones.py`, pon el cursor en `multiplicar`.
3. `C-c I x`.
4. Aplica el parche.
5. `C-c P t`.

Hecho cuando: los 8 tests pasan.

Ejercicio 5 — Refactor con diff (15 min)

Objetivo: aceptar un cambio sin romper nada.

1. Selecciona todo `calc/operaciones.py` (`C-x h`).
2. `C-c I f`.
3. `C-c I D` para revisar el diff.
4. `C-c I a` para aceptar (o `C-c I K` si no te convence).
5. `C-c P t`.

Hecho cuando: los 8 tests siguen verdes después del refactor.

Ejercicio 6 — Docstrings y tipos (10 min)

Objetivo: dejar el módulo presentable.

1. `C-x h` en `calc/operaciones.py`.
2. `C-c I d`.
3. Aplica el código.
4. `C-c P l` para ver diagnósticos.
5. `C-c P t`.

Hecho cuando: cada función tiene docstring y type hints, y no hay errores nuevos de Pyright.

Ejercicio 7 — Kata cronometrado (10 min)

Objetivo: medir tu velocidad real con el ciclo completo.

1. Crea `~/src/ia-python-practica/kata.py` con:

```python
def fizzbuzz(n):
    raise NotImplementedError
```

2. Crea `tests/test_kata.py` con:

```python
from kata import fizzbuzz

def test_fizzbuzz():
    assert fizzbuzz(3) == "Fizz"
    assert fizzbuzz(5) == "Buzz"
    assert fizzbuzz(15) == "FizzBuzz"
    assert fizzbuzz(7) == "7"
```

3. Cronómetro en marcha. Ciclo: `C-c P t` (rojo) → `C-c I i` → pegar → `C-c P f` → `C-c P t` (verde).
4. Anota el tiempo.

Hecho cuando: verde en menos de 10 minutos. Si tardas más, identifica en qué paso perdiste tiempo (leer, pegar, depurar) y repite el ejercicio.

Ejercicio 8 — Precisión (20 min)

Objetivo: mejorar sin cambiar el comportamiento.

1. `C-x h` en `calc/operaciones.py`.
2. `C-c I r`.
3. Elige **dos** hallazgos del review.
4. Aplica el parche mínimo de cada uno, uno por vez, con `C-c P t` entre ambos.
5. Mira el diff final (`C-x v =` si hay git, o `C-/` para comparar).

Hecho cuando: dos mejoras aplicadas, tests verdes y un diff pequeño y legible.

---

11. Git y GitHub con Magit

Magit convierte Git en una interfaz de una tecla por acción. No memorizas `git add -p` ni `git push -u origin main`: ves el estado, marcas lo que quieres y commiteas.

11.1 Preparación (una sola vez)

Forge necesita un token de GitHub. Configúralo con `./scripts/setup-forge.sh`, que usa el token de `gh` y crea `~/.authinfo` con permisos `600`. Si algún día caduca:

```
umask 077
LOGIN=$(gh api user -q .login); TOKEN=$(gh auth token)
{
  printf 'machine api.github.com login %s^forge password %s\n' "$LOGIN" "$TOKEN"
  printf 'machine api.github.com login %s^ghub password %s\n' "$LOGIN" "$TOKEN"
} > ~/.authinfo
chmod 600 ~/.authinfo
```

> **Aviso:** Las entradas deben llevar el sufijo `^forge` y `^ghub`; con solo `login usuario` Forge no encuentra el token.

En Magit, `'` abre Forge. La primera vez: **Add repository** y luego **Pull** para traer issues y PRs.

11.2 El buffer de Magit

`C-x g` (o `C-c G s`) abre el estado del repo. Verás:

- **Untracked files**: archivos que Git aún no conoce.
- **Unstaged changes**: cambios sin preparar.
- **Staged changes**: lo que entrará en el próximo commit.

`TAB` expande o colapsa; `RET` muestra el diff de un archivo.

11.3 Lo diario

Quiero…	En Magit	Global
Preparar archivo o hunk	s	—
Quitar de preparado	u	—
Descartar un cambio	x (confirma)	—

Quiero…	En Magit	Global
Commit	`c c`	`C-c G c`
Push	`P p`	`C-c G p`
Push fijando upstream	`P u`	—
Fetch	`f f`	`C-c G f`
Pull	`F u`	—
Log	`l l`	`C-c G l`
Diff de lo no preparado	`d d`	`C-c G d`
Cambiar de rama	`b b`	`C-c G b`
Crear rama	`b c`	—
Stash	`z z`	`C-c G S`
Blame	`M-x magit-blame`	`C-c G B`

En el buffer de commit: escribe el mensaje y `C-c C-c` para confirmar; `C-c C-k` para abortar.

> **Aviso:** Prepara por hunks (s sobre el trozo), no el archivo entero. Así cada commit cuenta una sola historia.

11.4 Ramas y pull requests con Forge

1. `b c` crea una rama (por ejemplo `feature/potencia`).
2. Haz cambios, `s`, `c c`, `P u` (la primera vez).
3. `'` → **Create pull-request** (Forge) o `M-x compile RET gh pr create --fill RET`.
4. En la lista de PRs: `RET` abre el PR, `c` comenta, `a` aprueba, `r` pide cambios.

Forge también lista **issues**: `'` → **Issues** → `RET` para leer, `c` para comentar, `C-c C-c` para enviar.

11.5 Enlaces y cambios en el margen

- `C-c G g` (`git-link`) copia la URL de GitHub de la línea, región o commit actual. Ideal para compartir un hallazgo.
- `diff-hl` marca en el margen las líneas añadidas, modificadas y borradas. `M-x diff-hl-next-hunk` y `M-x diff-hl-previous-hunk` saltan entre cambios.
- `C-c G B` abre el blame de la línea: quién cambió qué y cuándo.

11.6 Conflictos

Cuando un merge o rebase deja conflictos:

1. Magit marca los archivos en **Unmerged**.
2. `M-x smerge-mode` activa la resolución de conflictos.
3. `C-c ^ n` / `C-c ^ p` saltan al siguiente/anterior conflicto.
4. Elige una versión:
   - `M-x smerge-keep-mine` (tu versión, la de arriba).
   - `M-x smerge-keep-other` (la otra, la de abajo).
   - `C-c ^ a` conserva ambas.
   - `C-c ^ RET` conserva la del cursor.
5. `s` prepara el archivo resuelto y `c c` termina el merge.

Para verlo lado a lado: `M-x magit-ediff-dwim`.

11.7 GitHub más allá de Magit

`gh` sigue siendo útil para lo que Forge no cubre:

```
gh pr checks              # estado de CI del PR actual
gh run list               # últimas ejecuciones de Actions
gh release create v0.1.0  # crear un release
gh repo view --web        # abrir el repo en el navegador
```

Desde Emacs: `M-x compile RET gh pr checks RET`.

11.8 Deshacer con Git

Antes de experimentar, recuerda: **un commit es un punto de retorno**. Si algo sale mal, Git casi siempre puede volver atrás.

Quiero…	Comando	En Magit
Descartar cambios de un archivo	`git restore archivo`	x sobre el archivo/hunk
Descartar todo lo no preparado	`git restore .`	x en “Unstaged changes”
Quitar del stage sin perder cambios	`git restore --staged archivo`	u
Deshacer el último commit y conservar cambios	`git reset --soft HEAD~1`	—
Deshacer el último commit y los cambios	`git reset --hard HEAD~1`	—
Revertir un commit ya publicado	`git revert <hash>`	—
Recuperar un commit “perdido”	`git reflog` + `git reset --hard <hash>`	—
Guardar cambios temporalmente	`git stash push -m "wip"`	z z
Recuperar lo guardado	`git stash pop`	z p

> **Aviso:** `git reset --hard` borra los cambios no commiteados. Úsalo solo si estás seguro o si acabas de commitear.

Recuperar un commit borrado:

```
git reflog
# busca el hash del commit que quieres recuperar
git reset --hard <hash>
```

`git reflog` guarda el historial de movimientos de `HEAD` durante semanas. Es el “botón de deshacer” de Git.

Antes de un experimento riesgoso:

```
git switch -c experimento   # rama nueva
# o
git stash push -m "antes de probar"
```

Si el experimento sale bien, merge; si no, `git switch main` y borra la rama.

Ejercicios de Git

G1 — Primer commit y push (15 min)

1. Abre `~/src/ia-python-practica/`.
2. `C-x g` → verás los archivos sin trackear.
3. `S` para preparar todo (o `s` archivo por archivo).
4. `c c`, escribe el mensaje, `C-c C-c`.
5. `P p` para subir.

Hecho cuando: `gh repo view --web` muestra el commit en GitHub.

G2 — Rama y PR (20 min)

1. `b c` → `feature/potencia`.
2. Implementa `potencia` (ejercicio 3) y commitea.
3. `P u` para subir la rama.
4. `'` → **Create pull-request**, escribe título y descripción.
5. Vuelve a `main` (`b b`) y abre el PR desde `'` → **Pull requests**.

Hecho cuando: el PR existe en GitHub y Forge lo lista.

G3 — Conflicto (15 min)

1. En `main`, cambia una línea de `calc/operaciones.py` y commitea.
2. En la rama, cambia la misma línea y commitea.
3. `b b` → `main`, luego `M-x magit-merge` → `feature/potencia`.
4. Resuelve con `M-x smerge-mode` y `M-x smerge-keep-mine` / `smerge-keep-other`.
5. `s`, `c c`.

Hecho cuando: el merge termina y `C-c P t` sigue verde.

G4 — Revisar un PR (15 min)

1. `'` → **Pull requests** → elige el PR de G2.
2. `RET` para leer el diff.
3. `c` para comentar; `C-c C-c` para enviar.
4. `a` para aprobar (o `r` para pedir cambios).

Hecho cuando: el comentario o review aparece en GitHub.

G5 — Viaje en el tiempo (10 min)

1. `C-c G l` → log; `RET` sobre un commit para verlo.
2. `C-c G d` → diff de lo no preparado.
3. `C-c G B` → blame de una línea.
4. `C-c G g` → copia el enlace de esa línea y pégalo en el navegador.

Hecho cuando: puedes reconstruir quién cambió una línea y compartir el enlace exacto.

---

12. Proyectos: agente de código con DeepSeek

Hasta ahora la IA veía lo que le pegabas. Con el **agente de proyecto**, DeepSeek puede explorar el proyecto por su cuenta: listar archivos, leerlos, buscar patrones, editar con diff, ejecutar pytest y operar con Git. Todo dentro de la raíz del proyecto y con confirmación en cada cambio.

12.1 Abrir el agente

Abre un archivo del proyecto y pulsa:

`C-c I p`

Se abre `*IA: proyecto*`. La barra superior muestra la raíz y las herramientas. Escribe tu petición y `C-c RET`.

> **Aviso:** El agente fija la raíz del proyecto actual (`project.el`). Ábrelo desde un archivo dentro del proyecto; si lo abres desde otra carpeta, la raíz será la de esa carpeta.

12.2 Herramientas

Herramienta	Qué hace	Confirmación
`proyecto_arbol`	Lista archivos (rutas relativas)	No
`proyecto_leer`	Lee un archivo de texto	No
`proyecto_buscar`	Busca con `rg` (número de línea)	No
`proyecto_escribir`	Reescribe un archivo completo	Diff + `y/n`
`proyecto_reemplazar`	Reemplaza un fragmento exacto	Diff + `y/n`
`proyecto_crear`	Crea un archivo nuevo	Vista previa + `y/n`
`proyecto_borrar`	Mueve a la papelera	`y/n`
`proyecto_pytest`	Ejecuta pytest	No
`proyecto_git`	`status`, `diff`, `log`, `show`, `branch`	No
`proyecto_commit`	`git add` + `git commit`	Diff + `y/n`
`proyecto_push`	`git push`	Commits + `y/n`

> **Importante:** El agente no tiene shell libre: solo pytest y los comandos Git de la lista. No puede ejecutar `rm`, `curl` ni nada arbitrario.

12.3 El ciclo del agente

```
1. "Lee calc/operaciones.py y dime qué falla"
   → proyecto_leer
2. "Reemplaza la suma por una multiplicación en multiplicar"
   → proyecto_reemplazar → diff → y
3. "Ejecuta los tests"
   → proyecto_pytest → 3 fallos, 5 aprobados
4. "Muéstrame el diff y haz commit"
```

```
   → proyecto_git diff → proyecto_commit → diff → y
5. "Sube los cambios"
   → proyecto_push → commits → y
```

Cada paso es una petición normal en el chat. No escribes comandos: el modelo elige la herramienta.

12.4 Ejemplos de peticiones

- "Explora el proyecto y dime qué archivos hay."
- "Busca dónde se usa `promedio`."
- "Lee `calc/operaciones.py` y explica el bug de `multiplicar`."
- "Reemplaza `return a + b` por `return a * b` en `multiplicar`."
- "Ejecuta pytest y dime qué falla."
- "Corrige el test que falla y vuelve a ejecutar."
- "Muéstrame `git diff` y propone un mensaje de commit."
- "Haz commit y push."

Ejercicios del agente

P1 — Explorar (10 min)

1. Abre `~/src/ia-python-practica/calc/operaciones.py`.
2. `C-c I p`.
3. Pide: "Explora el proyecto, lista los archivos y busca `promedio`."
4. Comprueba que usó `proyecto_arbol` y `proyecto_buscar`.

Hecho cuando: la respuesta menciona `calc/operaciones.py`, `tests/test_operaciones.py` y las líneas de `promedio`.

P2 — Editar con diff (15 min)

1. En el mismo chat: "Reemplaza en `calc/operaciones.py` el `return a + b` de `multiplicar` por `return a * b`."
2. Revisa el diff en `*IA: cambio propuesto*`.
3. Responde `y`.
4. Pide: "Ejecuta pytest."
5. Deben quedar **3 fallos** (`promedio`, `promedio_vacio`, `potencia`).

Hecho cuando: el diff era correcto y pytest ya no falla en `multiplicar`.

P3 — Ciclo completo (20 min)

1. Pide: "Implementa `promedio` y `potencia` para que pasen sus tests."
2. Revisa cada diff y acepta.
3. Pide: "Ejecuta pytest."
4. Pide: "Muéstrame `git diff` y haz commit con un mensaje claro."
5. Revisa el diff del commit y acepta.

Hecho cuando: 8 tests verdes y un commit nuevo en `git log`.

P4 — Push (10 min)

1. Pide: "Haz push."

2. Revisa los commits pendientes y confirma.
3. Verifica en GitHub.

Hecho cuando: `gh repo view --web` muestra el commit.

12.5 Seguridad

- La raíz se fija al abrir el agente; las rutas fuera del proyecto se rechazan.
- Escrituras y borrados siempre con diff/confirmación.
- Commits y push con confirmación.
- Sin shell libre.
- Las claves nunca se pasan al modelo: viven en `~/.emacs.d/.deepseek-key` y `~/.authinfo`.

Si algo sale mal, `C-/` deshace en el buffer y Git te permite volver atrás (sección 15).

---

13. Chuleta de una página

```
┌─ BÁSICO ──────────────────────────────────────────────────┐
│ C-x C-f  abrir      C-x C-s  guardar    C-x C-c  salir    │
│ C-SPC    marcar     C-x h    todo       C-g      cancel   │
│ C-/      deshacer   C-h k    ¿qué tecla?                  │
├─ IA (C-c I) ──────────────────────────────────────────────┤
│ e explicar   t tests      i implementar   r revisar       │
│ f refactor   d docstrings x corregir      c chat          │
│ p agente     a aceptar    K rechazar      D diff          │
│ m menú       1 DeepSeek   2 V4 Pro        3 Gemini        │
│ s enviar                                                  │
├─ PYTHON (C-c P) ──────────────────────────────────────────┤
│ r ejecutar   t pytest     f black         l lint          │
│ c REPL                                                    │
├─ GIT (C-c G) ─────────────────────────────────────────────┤
│ s status     c commit     p push          f fetch         │
│ l log        d diff       b rama          B blame         │
│ S stash      g enlace     F Forge                         │
└───────────────────────────────────────────────────────────┘
```

El ciclo: `C-c P t` → `C-c I i` → pegar → `C-c P f` → `C-c P t` → `C-c I r`.

---

14. Problemas comunes

`C-c I` no hace nada. Es un prefijo: pulsa la segunda tecla (`C-c I e`). Comprueba con `C-h k C-c I e`.

La respuesta no llega. Mira el buffer `*IA: ...*` (puede estar detrás). Comprueba el modelo con `C-c I 1`. Si sigue, revisa la clave en `~/.emacs.d/.deepseek-key`.

`pytest` no se encuentra. En terminal: `python3 -m pytest --version`. Si falla, instala pytest. El comando usa `python3 -m pytest`, no el binario suelto.

`ModuleNotFoundError: calc`. Ejecuta desde la raíz del proyecto. `pyproject.toml` ya incluye `pythonpath = ["."]`; si lo moviste, vuelve a ponerlo.

LSP tarda o no responde. `M-x lsp-workspace-restart`. La primera apertura de un proyecto es la más lenta.

`black` no formatea. Comprueba `black --version`. Si existe, `M-x blacken-buffer` a mano para ver el error.

Acepté un refactor y rompí todo. `C-/` repetido hasta volver al estado anterior. Después, `C-c P t` para confirmar.

La respuesta se corta. El modelo llegó a su límite. Escribe "continúa" en el buffer y `C-c RET`.

Quiero ver qué modelo está activo. `C-c I m` abre el menú con el backend y el modelo actuales.

`C-x g` dice que no hay repositorio. Abre un archivo dentro del repo y vuelve a `C-x g`. Magit usa el `default-directory` del buffer actual.

Forge pide token o no lista issues/PRs. Revisa que `~/.authinfo` tenga las dos entradas (`^forge` y `^ghub`) y permisos `600`. Luego, en Magit: `'` → **Add repository** y **Pull**.

El push es rechazado. `f f` para traer cambios, `F u` para integrarlos, y `P p` de nuevo. Si es la primera vez de la rama, `P u` fija el upstream.

Hay conflictos y no sé qué hacer. `M-x smerge-mode`, `C-c ^ n` para navegar y `M-x smerge-keep-mine` / `smerge-keep-other`. Para verlo lado a lado: `M-x magit-ediff-dwim`.

`diff-hl` no se ve. `M-x global-diff-hl-mode`. En terminal sin margen gráfico, `M-x diff-hl-margin-mode`.

El agente dice que la ruta está fuera del proyecto. Ábrelo con `C-c I p` desde un archivo dentro del proyecto. La raíz se fija al abrir el agente; si lo abres desde otra carpeta, la raíz será esa otra carpeta.

El agente no usa las herramientas. Comprueba que estás en `*IA: proyecto*` (no en un buffer `*IA: explicar*`). Las herramientas son locales a ese buffer. Pídele explícitamente: "usa `proyecto_leer` para leer `calc/operaciones.py`".

El agente pide confirmación y no veo el diff. El diff se abre en `*IA: cambio propuesto*`. Si no lo ves, usa `C-x b` y elige ese buffer. La pregunta sale en el minibuffer: responde `y` o `n`.

El commit falla con "No hay cambios para commitear". El agente hace `git add -A` antes de commitear; si no hay cambios, no hay commit. Revisa `C-c G s`.

El push falla. Comprueba el remoto y el upstream con `C-c G s`. La primera vez, `git push -u origin HEAD` (el agente lo hace solo si no hay upstream).

Quiero deshacer lo que hizo el agente. `C-/` en el buffer para la última edición; para todo lo demás, `git restore`, `git reset` o `./scripts/reset.sh` (sección 15).

Para ir más lejos

- **Contexto de archivos:** en un buffer de gptel, `M-x gptel-context-add-file` añade un archivo entero al contexto.
- **Org-babel:** en un `.org`, un bloque `#+begin_src python` se ejecuta con `C-c C-c`. Útil para prototipos y gráficas.
- **Jupyter:** `jupyter-run-repl` abre un REPL de Jupyter si necesitas kernels.
- **Directivas:** `C-c I m` permite elegir la directiva `programador` (tu prompt maestro) o crear otras.
- **Forge:** además de PRs e issues, `'` permite listar repositorios, notificaciones y temas asignados.
- **gh desde Emacs:** `M-x compile` con cualquier comando `gh` (CI, releases, gists) sin salir del editor.
- **Agente de proyecto:** `C-c I p` combina exploración, edición con diff, pytest y Git en un solo chat.
- **Configuración pública:** la guía y una config de Emacs sin claves están en `https://github.com/lmedinar/emacs-deepseek-assistance-tools`.

  Dato: Configuración verificada: gptel 20261018.848, DeepSeek `deepseek-flash` (V4.1 Flash) y `deepseek-v4-pro`, Pyright, pytest 9.1.1, black, Magit, Forge, el agente de proyecto y `gh` autenticado con `gh`.

---

15. Repetir la guía desde cero

El proyecto de práctica tiene una etiqueta `estado-inicial` que apunta al commit con el código roto. Para volver a empezar:

```
cd ~/src/ia-python-practica
./scripts/reset.sh
```

El script hace:

```
git reset --hard estado-inicial
git clean -fdx
```

`git reset --hard` devuelve los archivos versionados al commit; `git clean -fdx` borra los no versionados (incluidos `__pycache__`, `.pytest_cache` y cualquier archivo que hayas creado). Después verás de nuevo **4 fallos, 4 aprobados**.

Si el tag no existe:

```
git tag -f estado-inicial
```

Si algo sale mal

Situación	Solución
Edité un archivo y quiero volver	`git restore archivo`
Edité todo y quiero volver	`git restore .`
Hice commit y quiero deshacerlo (conservando cambios)	`git reset --soft HEAD~1`
Hice commit y quiero borrarlo todo	`git reset --hard HEAD~1`
Borré un commit por error	`git reflog` y `git reset --hard <hash>`
Quiero guardar cambios sin commitear	`git stash push -m "wip"` / `git stash pop`
Quiero volver al estado inicial	`./scripts/reset.sh`

> **Aviso:** Si `git status` muestra archivos que no quieres, `git clean -nd` te dice qué borraría `git clean -fd` sin borrar nada. Revisa antes de ejecutar.

Versión pública

La guía y una configuración de Emacs sin claves están en:

`https://github.com/lmedinar/emacs-deepseek-assistance-tools`

No incluyas nunca tus claves en un repositorio. La config pública usa `DEEPSEEK_API_KEY` o `~/.emacs.d/.deepseek-key`, y `~/.authinfo` para GitHub.